UNIT- V
MINIMUM SPANNING TREE AND SINGLE SOURCE SHORTEST PATH

OUTLINE

- Growing a minimum spanning tree
- The algorithms of Kruskal and Prim
- The Bellman-Ford algorithm
- Single-source shortest paths in directed acyclic graphs
- Dijkstra's algorithm
- Difference constraints and shortest paths
- Proofs of shortest-paths properties

INTRODUCTION

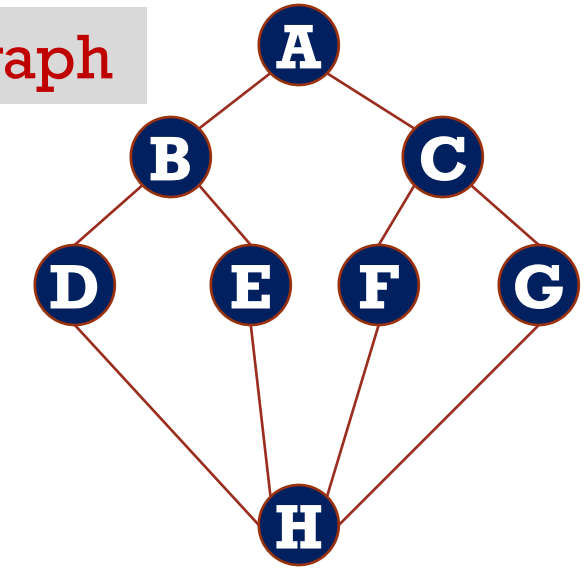
INTRODUCTION TO MINIMUM SPANNING TREE (MST)

- Let $G = \langle N, (A \rangle)$ be a **connected, undirected graph** where,
  1. N is the set of nodes and
  2. A is the set of edges.
- Each edge has a given **positive length or weight**.
- A spanning tree of a graph G **is a sub-graph** which is basically a tree and it contains all the vertices of G but does not contain cycle.
- A minimum spanning tree (MST) of a **weighted connected graph** G is a spanning tree with minimum or smallest weight of edges.
- Two Algorithms for **constructing** minimum spanning tree are,
  1. Kruskal's Algorithm
  2. Prim's Algorithm

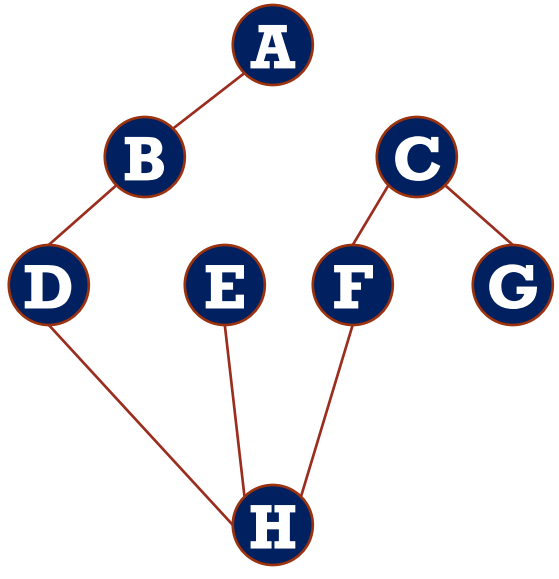
MINIMUM SPANNING TREE

SPANNING TREE EXAMPLES

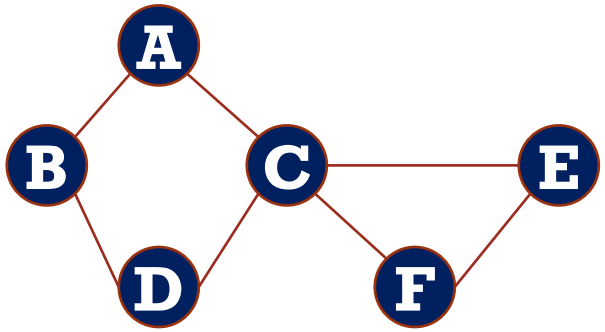
Graph



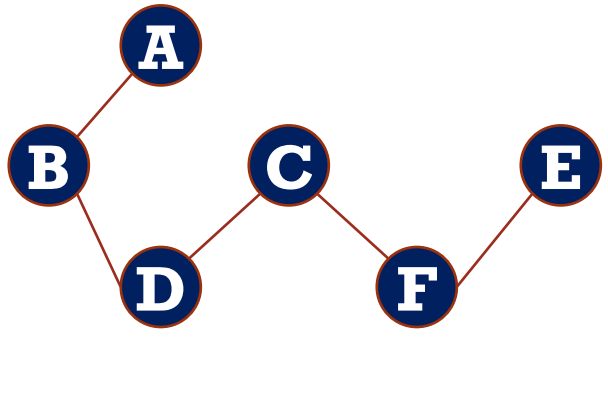
Spanning Tree



Graph

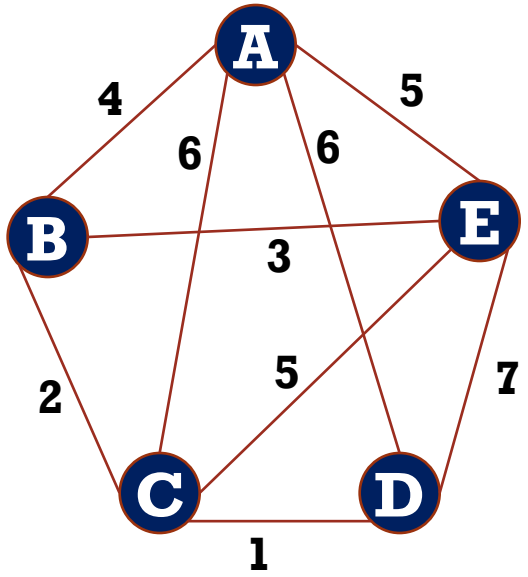


Spanning Tree



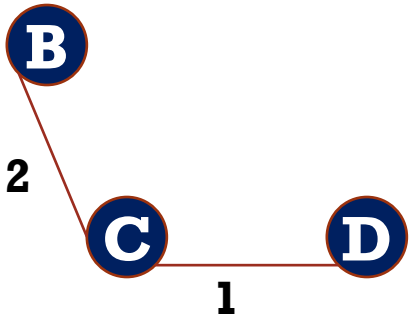
KRUSKAL'S ALGORITHM

KRUSKAL'S ALGORITHM FOR MST – EXAMPLE 1

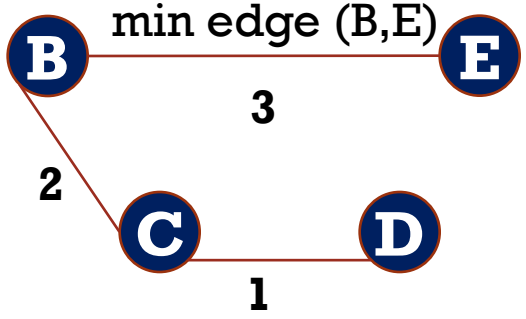


Step 1: Taking min edge (C,D)

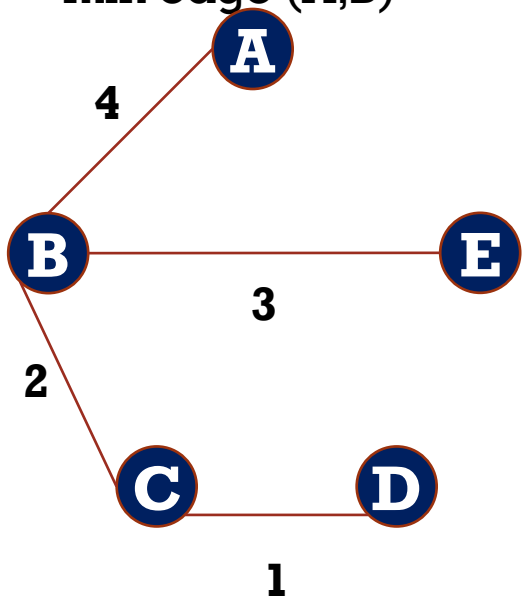
Step 2: Taking next min edge (B,C)



Step 3: Taking next min edge (B,E)



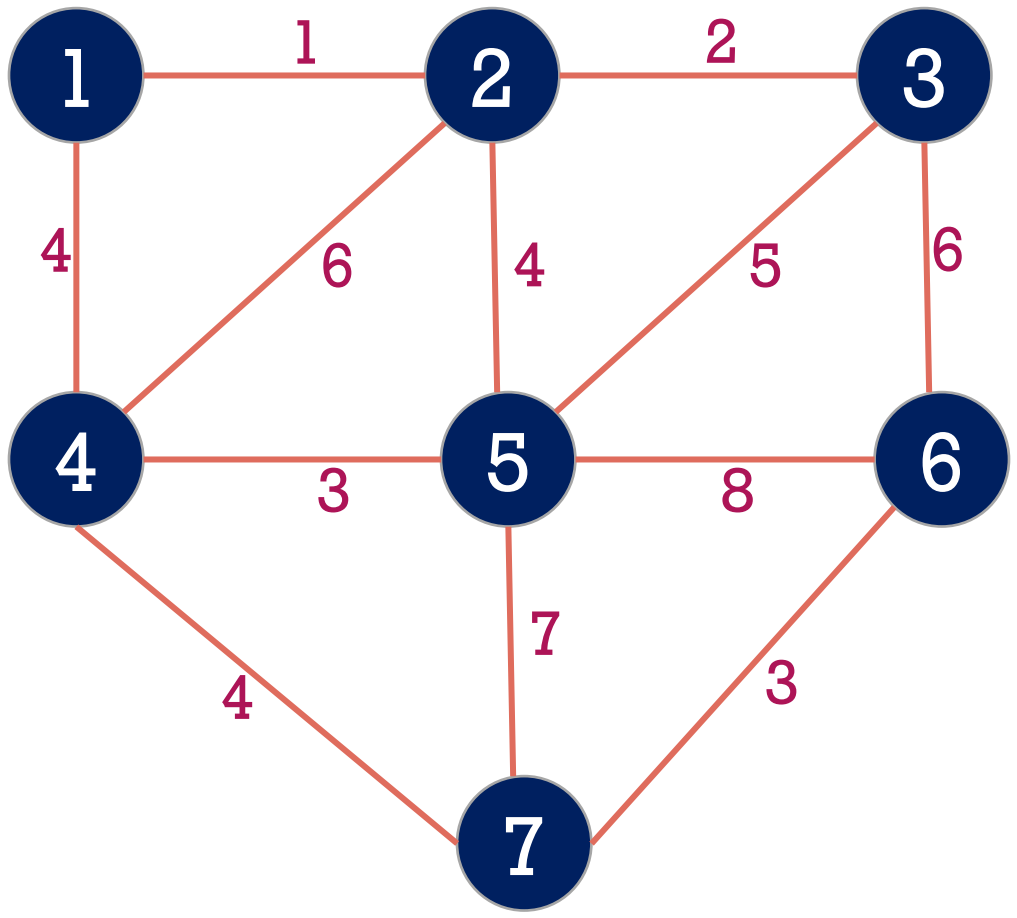
Step 4: Taking next min edge (A,B)



So, we obtained a minimum spanning tree of cost:

4 + 2 + 1 + 3 = 10

Kruskal's Algorithm for MST – Example 2

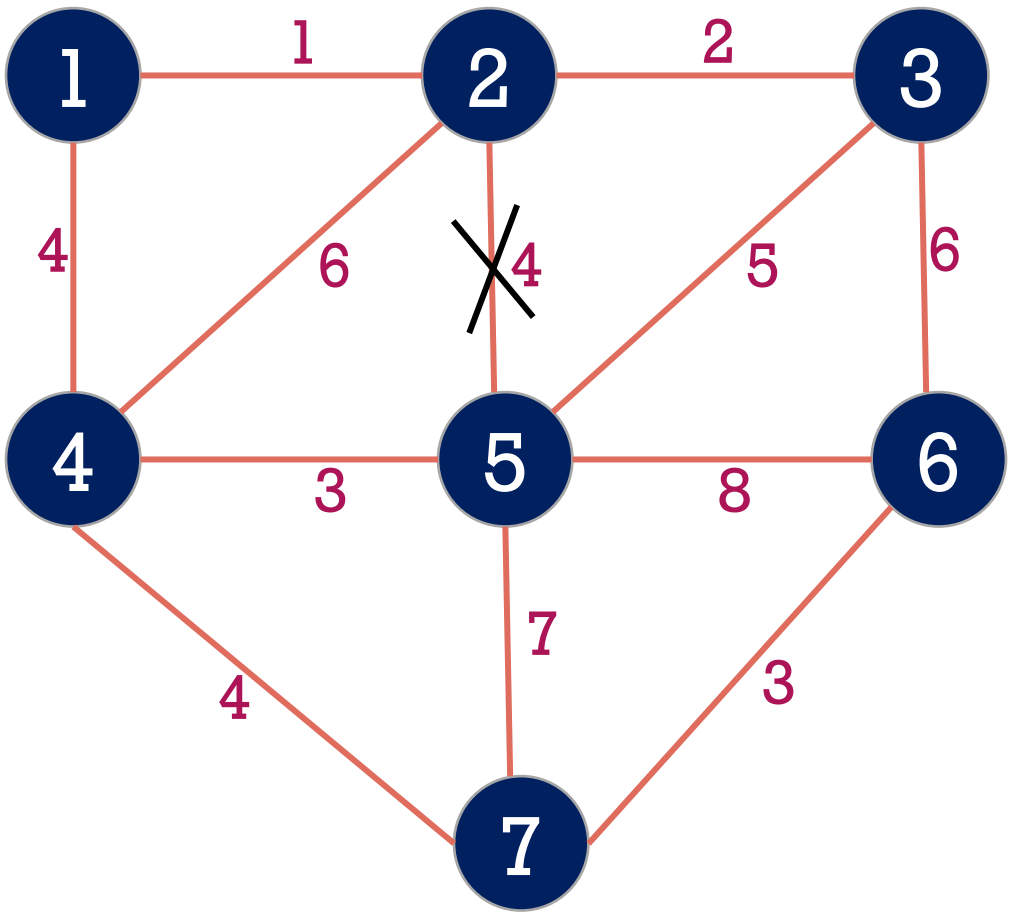


Step:1

Sort the edges in increasing order of their weight.

Edges	Weight	
{1, 2}	1	
{2, 3}	2	
{4, 5}	3	
{6, 7}	3	
{1, 4}	4	
{2, 5}	4	
{4, 7}	4	
{3, 5)	5	
{2, 4}	6	
{3, 6}	6	
{5, 7}	7	
{5, 6}	8	

Kruskal's Algorithm for MST – Example 2

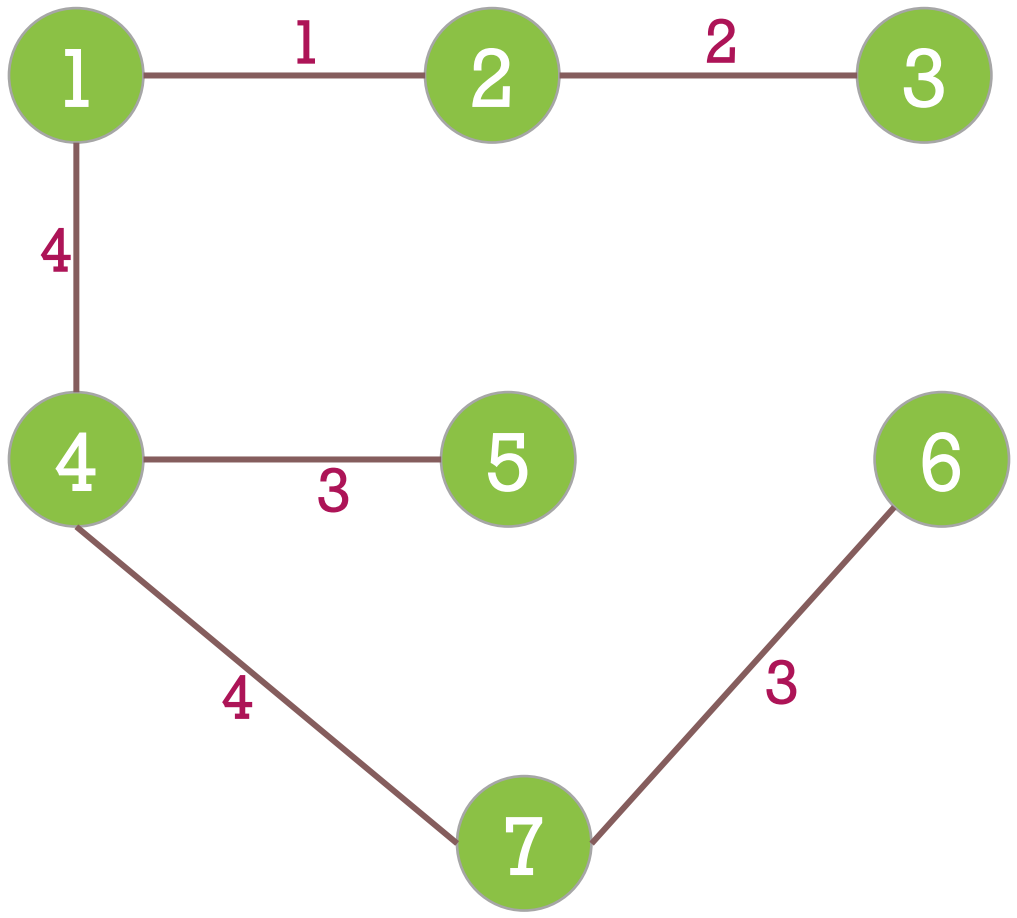


Step:2

Select the minimum weight edge but no cycle.

Edges	Weight	
{1, 2}	1	√
{2, 3}	2	√
{4, 5}	3	√
{6, 7}	3	√
{1, 4}	4	√
~~{2, 5}~~	~~4~~	
{4, 7}	4	√
{3, 5)	5	
{2, 4}	6	
{3, 6}	6	
{5, 7}	7	
{5, 6}	8	

Kruskal's Algorithm for MST – Example 2



Step:3 The minimum spanning tree for the given graph.

Edges	Weight	
{1, 2}	1	√
{2, 3}	2	√
{4, 5}	3	√
{6, 7}	3	√
{1, 4}	4	√
{4, 7}	4	√

Total Cost = 17

KRUSKAL'S ALGORITHM FOR MST

```
Function Kruskal(G = (N, A))
Sort A by increasing length
n ← the number of nodes in N
T ← Ø {edges of the minimum spanning tree}
Define n sets, containing a different element of set N
repeat
   e ← {u, v} //e is the shortest edge not yet considered
   ucomp ← find(u)
   vcomp ← find(v)
   if ucomp ≠ vcomp then merge(ucomp, vcomp)
   T ← T U {e}
until T contains n - 1 edges
return T
```

find(u) tells in which connected component a node u is found

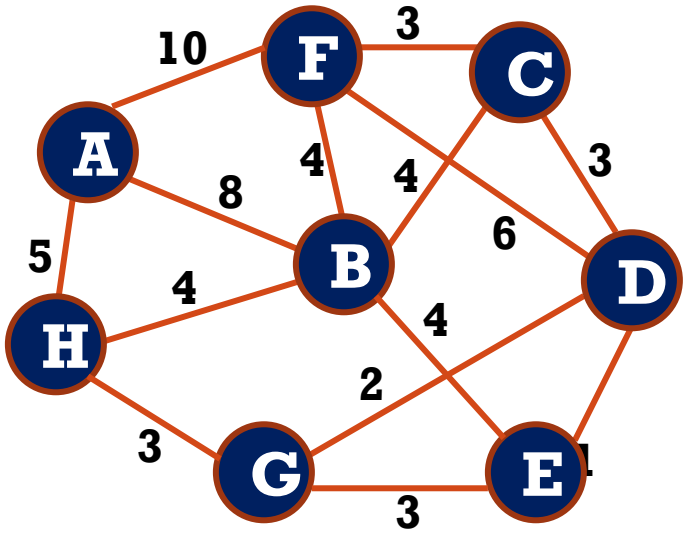
merge(ucomp, vcomp) is used to merge two connected components.

EXERCISES – HOME WORK

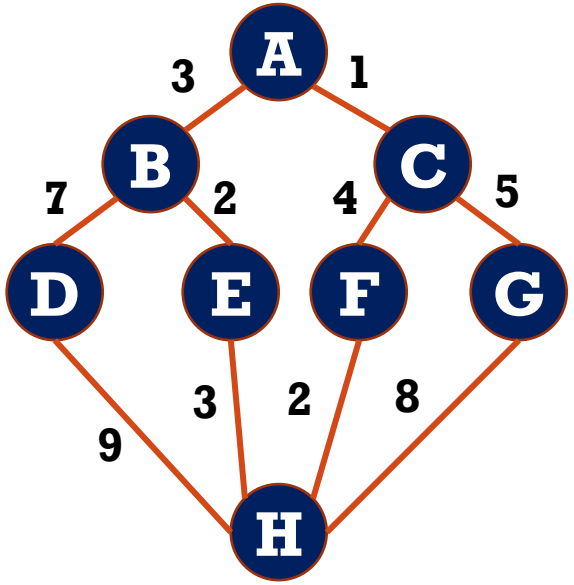
- The complexity for the Kruskal's algorithm is in $\theta(a \log n)$ where a is total number of edges and n is the total number of nodes in the graph G.

- Write the kruskal's Algorithm to find out Minimum Spanning Tree. Apply the same and find MST for the graph given below.

1.



2.

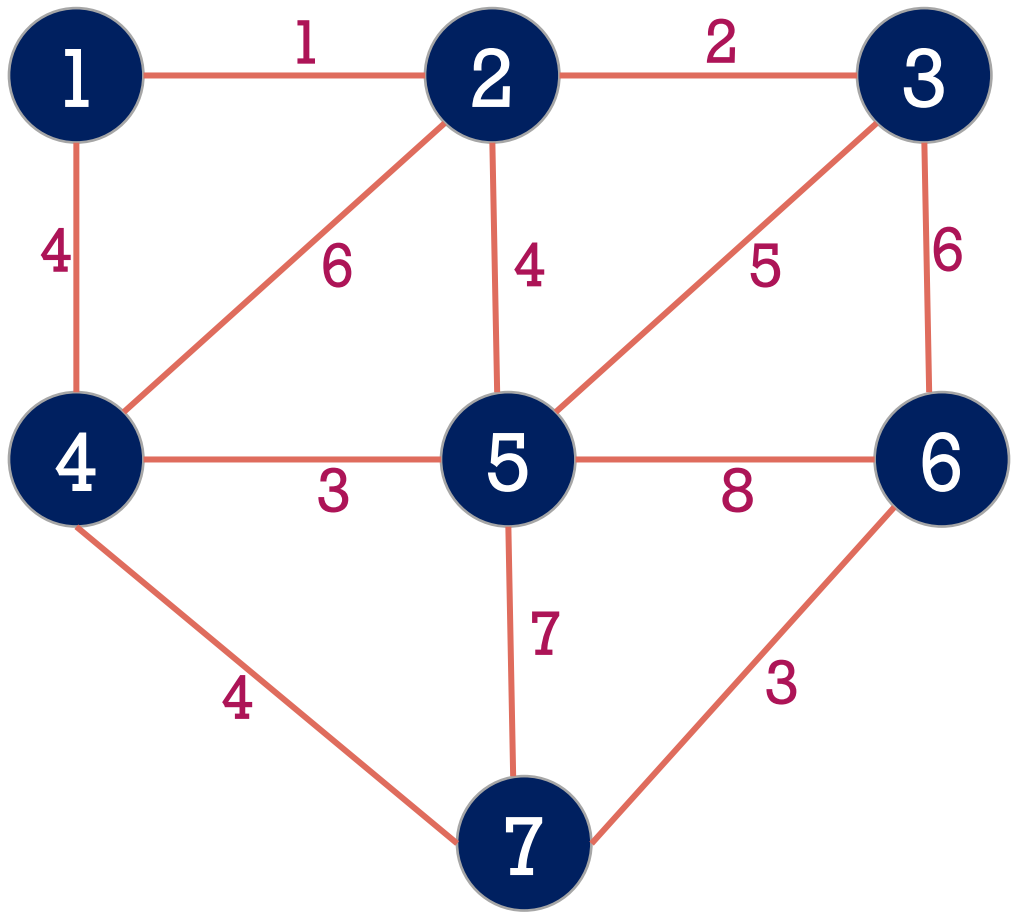


PRIM'S ALGORITHM

PRIM'S ALGORITHM

- In Prim's algorithm, the minimum spanning tree grows in a natural way, starting from an arbitrary root.
- At each stage we add a new branch to the tree already constructed; the algorithm stops when all the nodes have been reached.
- The complexity for the Prim's algorithm is $\boldsymbol{\theta(n^2)}$ where n is the total number of nodes in the graph G.

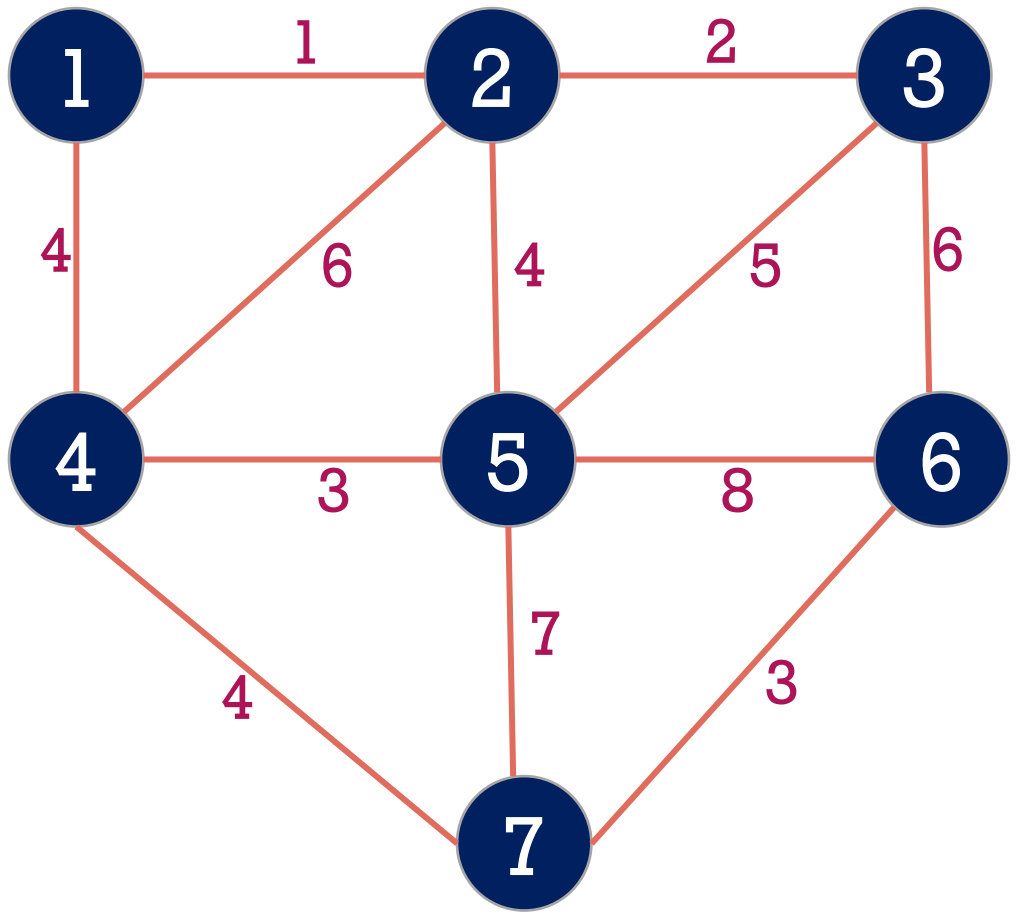
Prim's Algorithm for MST – Example 1



Step:1 Select an arbitrary node.

Node - Set B	Edges
1	

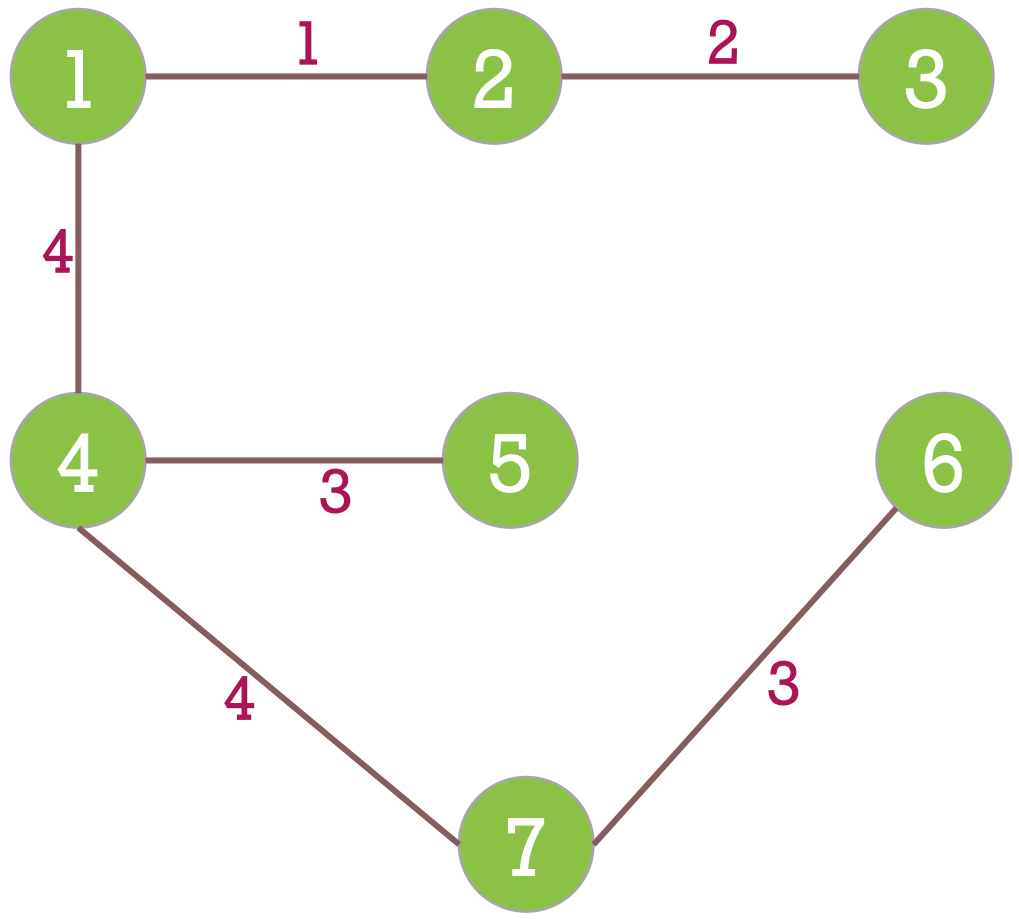
Prim's Algorithm for MST – Example 1



Step:2 Find an edge with minimum weight.

Node - Set B	Edges
1	{1, 2}, ~~{1, 4}~~
1, 2	~~{1, 4}~~, {2, 3} ~~{2, 4}~~, ~~{2, 5}~~
1, 2, 3	{1,4}, ~~{2,4}~~, ~~{2,5}~~, ~~{3,5}~~, {3,6}
1, 2, 3, 4	~~{2,4}~~ ~~{2,5}~~ ~~{3,5}~~ {3,6} ~~{4,5}~~ {4,7}
1, 2, 3, 4, 5	~~{2,4}~~ ~~{2,5}~~ ~~{3,5}~~ {3,6} ~~{4,7}~~ ~~{5,6}~~ {5,7}
1, 2, 3, 4, 5, 7	~~{2,4}~~ ~~{2,5}~~ ~~{3,5}~~ ~~{3,6}~~ ~~{5,6}~~ {5,7} {6,7}
1, 2, 3, 4, 5, 6, 7	

Prim's Algorithm for MST – Example 1



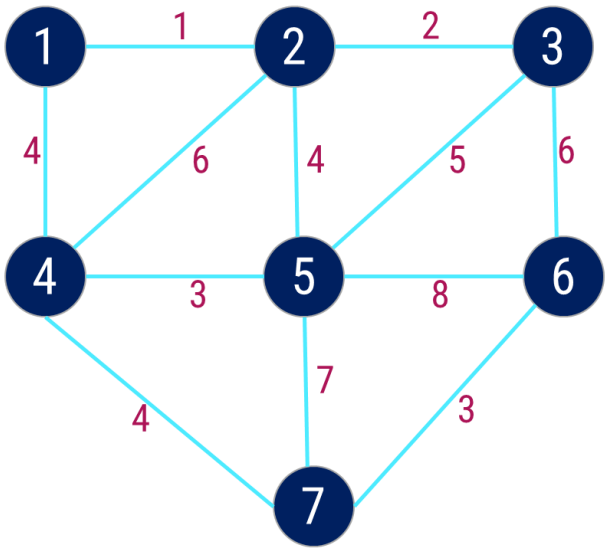
Step:3

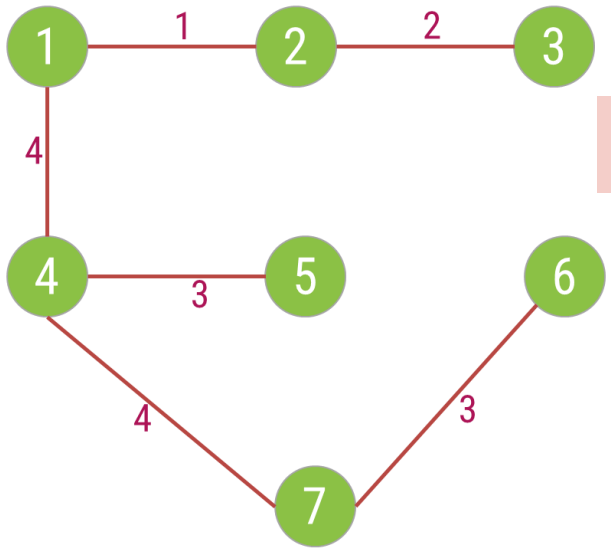
The minimum spanning tree for the given graph.

Node	Edges
1	
1, 2	{1, 2}
1, 2, 3	{2, 3}
1, 2, 3, 4	{1, 4}
1, 2, 3, 4, 5	{4, 5}
1, 2, 3, 4, 5, 7	{4, 7}
1, 2, 3, 4, 5, 6, 7	{6, 7}

Total Cost = 17

PRIM'S ALGORITHM – EXAMPLE 1





Cost = 17

Step	Edge Selected {u, v}	Set B	Edges Considered
Init.	-	{1}	--
1	{1, 2}	{1,2}	**{1,2}** {1,4}
2	{2, 3}	{1,2,3}	{1,4} **{2,3}** {2,4} {2,5}
3	{1, 4}	{1,2,3,4}	**{1,4}** {2,4} {2,5} {3,5} {3,6}
4	{4, 5}	{1,2,3,4,5}	{2,4} {2,5} {3,5} {3,6} **{4,5}** {4,7}
5	{4, 7}	{1,2,3,4,5,7}	{2,4} {2,5} {3,5} {3,6} **{4,7}** {5,6} {5,7}
6	{6,7}	{1,2,3,4,5,6,7}	{2,4} {2,5} {3,5} {3,6} {5,6} {5,7}

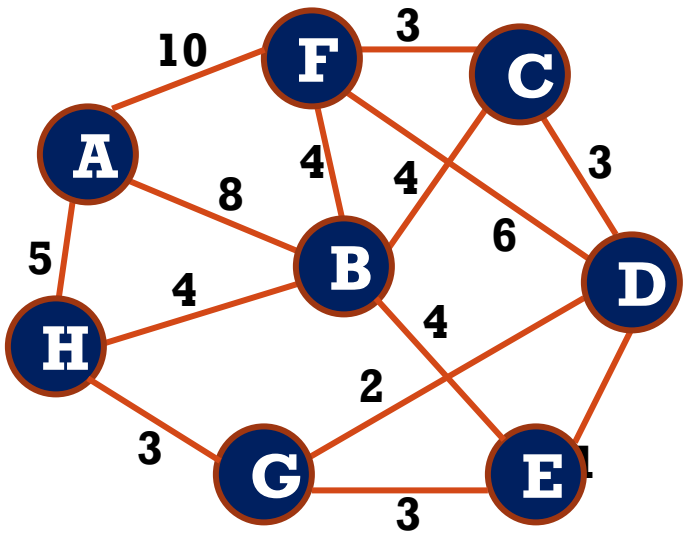
PRIM'S ALGORITHM

```
Function Prim(G = (N, A): graph; length: A → R+): set of edges
T ← ∅
B ← {an arbitrary member of N}
while B ≠ N do
      find e = {u, v} of minimum length such that
            u ∈ B and v ∈ N \ B
      T ← T U {e}
      B ← B U {v}
return T
```

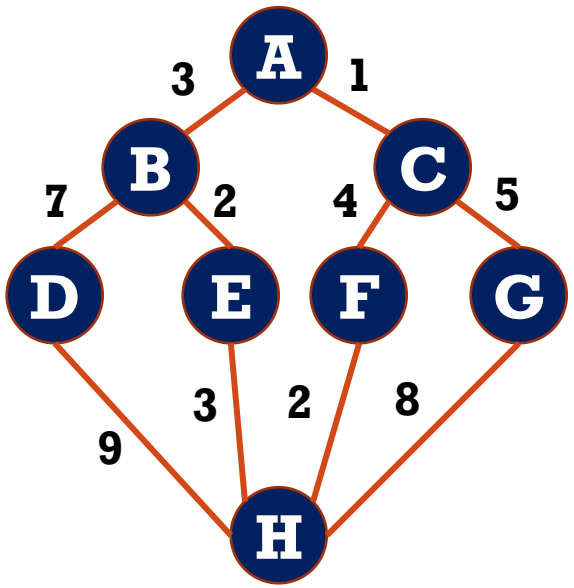
EXERCISES – HOME WORK

- Write the Prim's Algorithm to find out Minimum Spanning Tree. Apply the same and find MST for the graph given below.

1.



2.



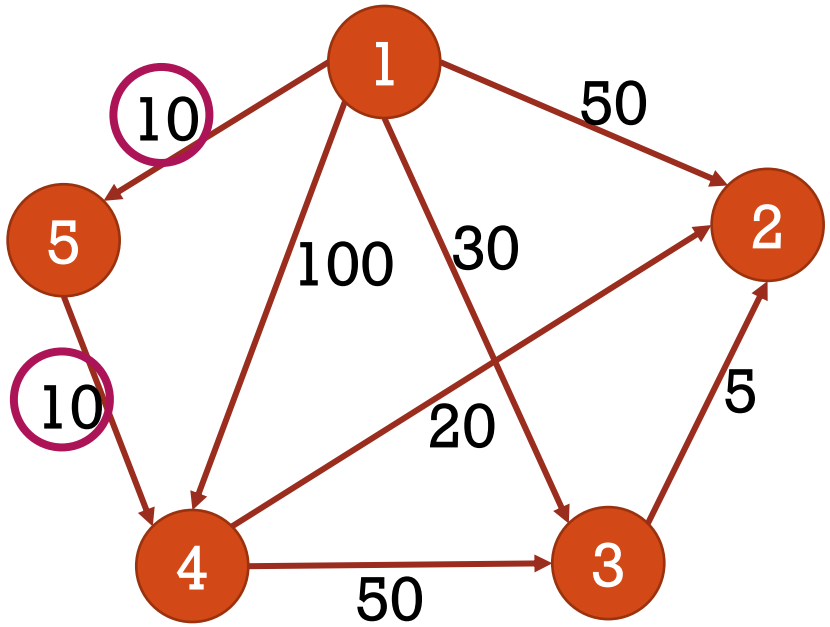
SINGLE SOURCE SHORTEST PATH – DIJKSTRA'S ALGORITHM

INTRODUCTION

- Consider now **a** directed graph $G = (N, A)$ where N is the set of nodes and A is the set of directed edges of graph G.
- Each edge has a positive length.
- One of the nodes is designated as the source node.
- The problem is to determine the length of the shortest path from the source to each of the other nodes of the graph.
- Dijkstra's Algorithm is for finding the shortest paths between the nodes in a graph.
- For a given source node, the algorithm finds the shortest path between the source node and every other node.
- The **algorithm maintains a matrix** $\boldsymbol{L}$ which gives the length of each directed edge:

$$L[i,j] \geq 0 \text{ if the edge } (i,j) \in A, \text{ and}$$
$$L[i,j] = \infty \text{ otherwise.}$$

DIJKSTRA'S ALGORITHM - EXAMPLE



Single source shortest path algorithm

Source node =

Step	v	C	2	3	4	5
Init.	-	{2, 3, 4, 5}	50	30	100	10
1	5	{2, 3, 4}	50	30	20	10

Is there path from 1 - 5 - 4

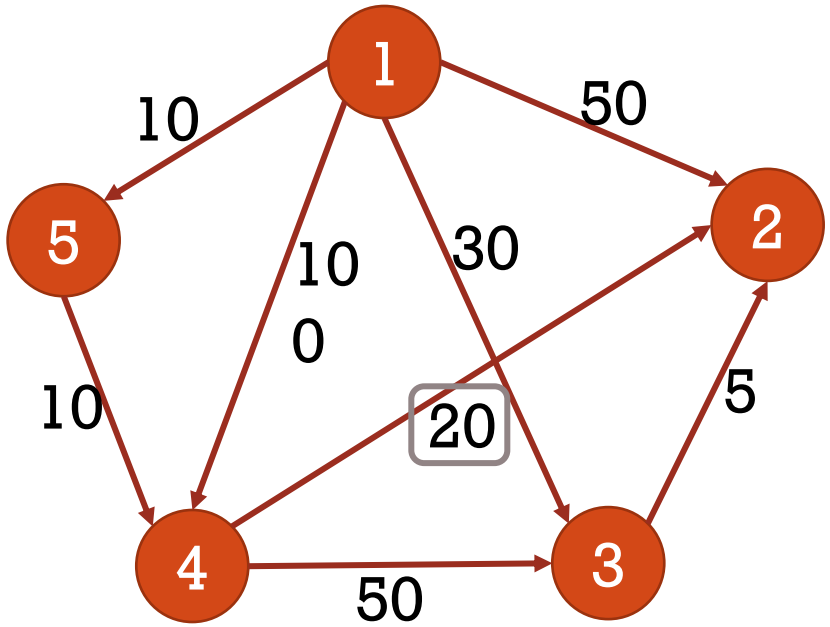
Is there path from 1 - 5 - 3

Is there path from 1 - 5 - 2

Yes

Compare cost of 1–5–4 (20) and 1–4 (100)

DIJKSTRA'S ALGORITHM - EXAMPLE



Single source shortest path algorithm

Source node =

Step	v	C	2	3	4	5
Init.	-	{2, 3, 4, 5}	50	30	100	10
1	5	{2, 3, 4}	50	30	20	10
2	4	{2, 3}	40	30	20	10

Is there path from 1 - 4 - 5

Is there path from 1 - 4 - 3

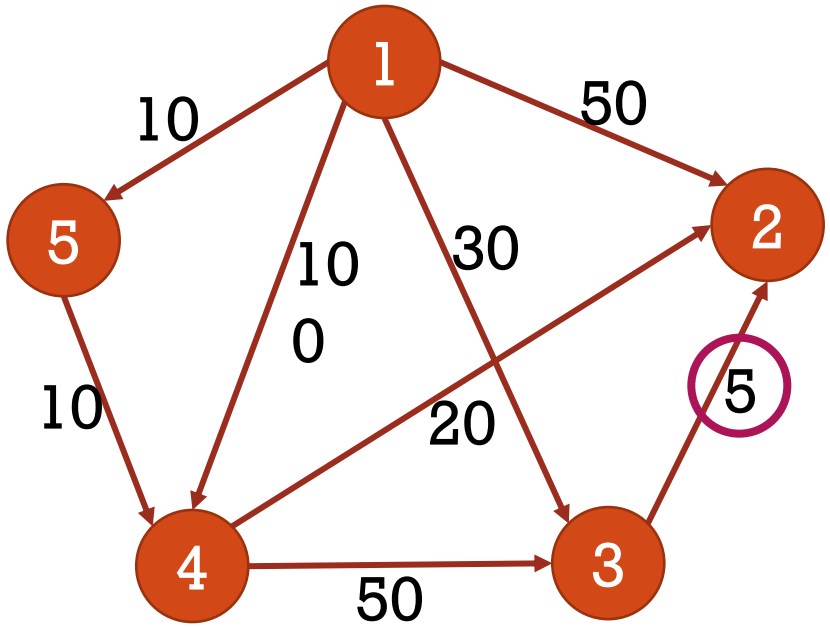
Is there path from 1 - 4 - 2

No

Compare cost of 1–4–3 (70) and 1–3 (30)

Compare cost of 1–4–2 (40) and 1–2(50)

DIJKSTRA'S ALGORITHM - EXAMPLE



Single source shortest path algorithm

Source node =

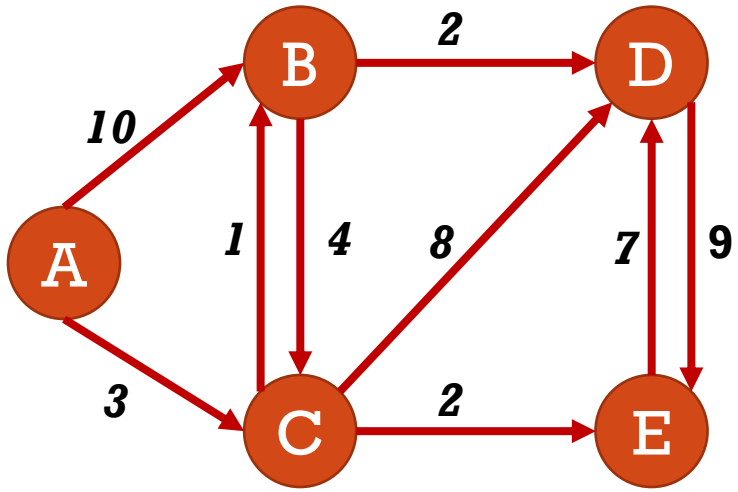
Step	v	C	2	3	4	5
Init.	-	{2, 3, 4, 5}	50	30	100	10
1	5	{2, 3, 4}	50	30	20	10
2	4	{2, 3}	40	30	20	10
3	3	{2}	35	30	20	10

f 1–3–2 and 1–2

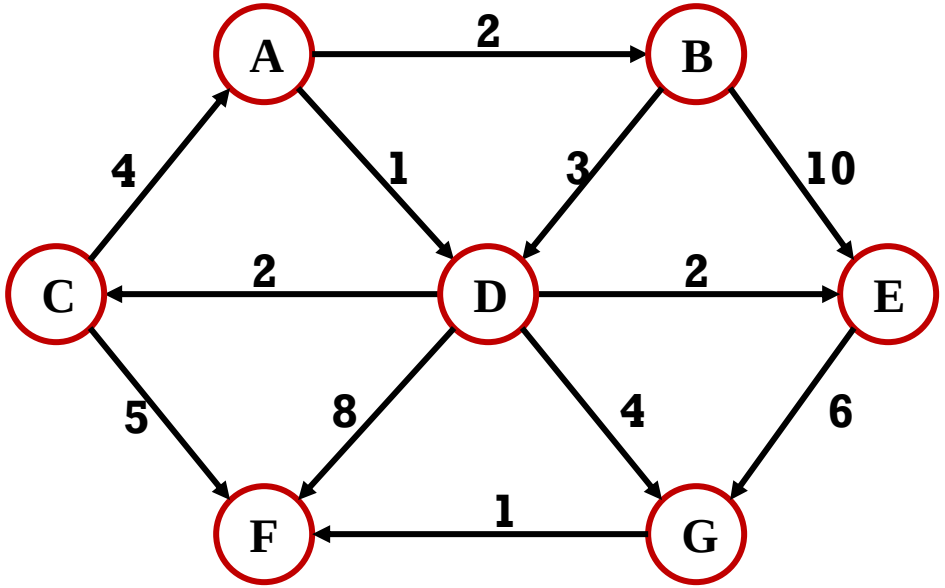
EXERCISES – HOME WORK

- Write Dijkstra's Algorithm for shortest path. Use the algorithm to find the shortest path from the following graph.

1.



2.

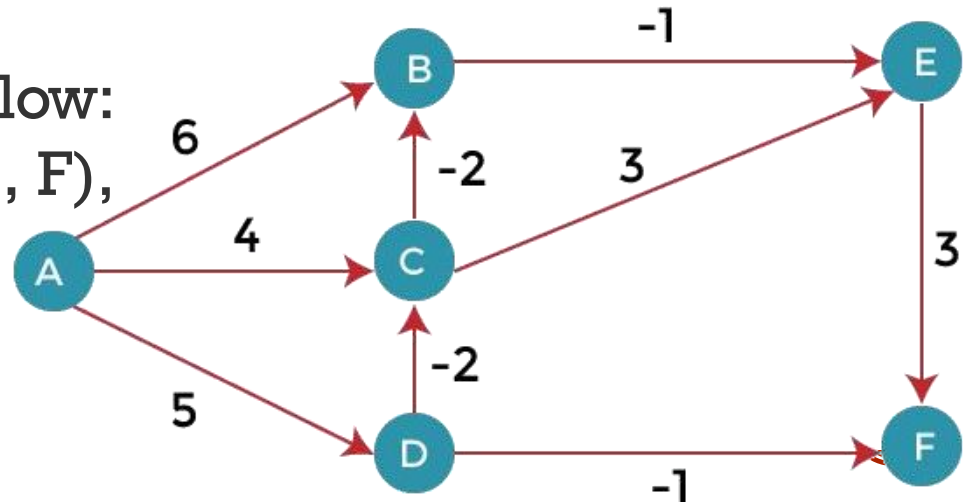


BELLAMAN- FORD ALGORITHM

BELLMAN FORD ALGORITHM

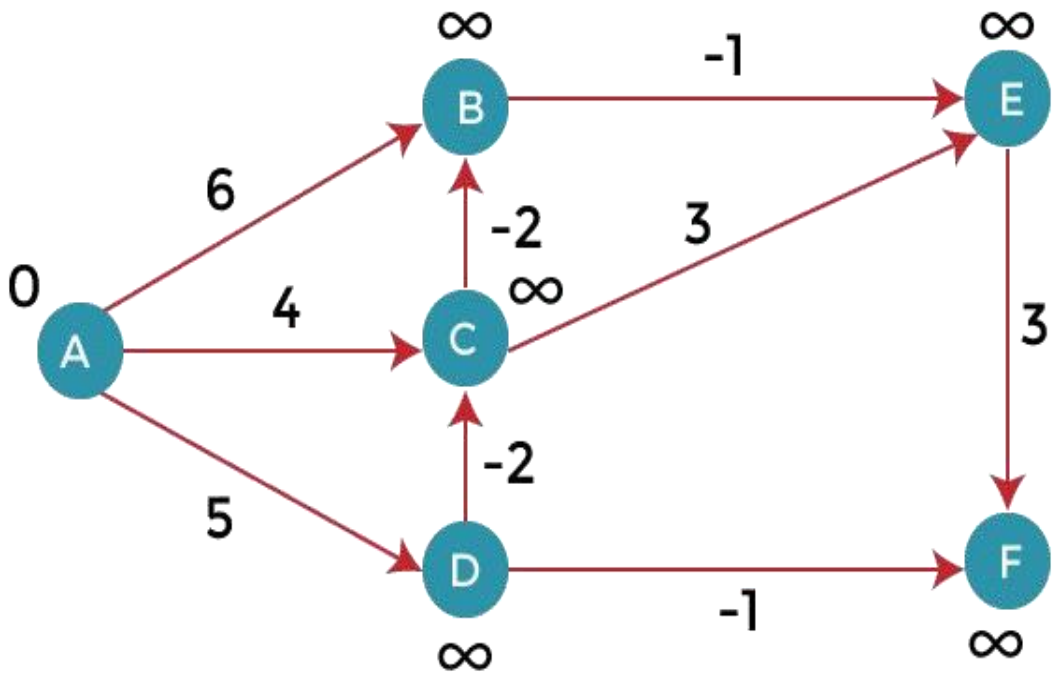
- Bellman ford algorithm is a single-source shortest path algorithm. This algorithm is used to find the shortest distance from the single vertex to all the other vertices of a weighted graph.
- In contrast to Dijkstra algorithm, bellman ford algorithm guarantees the correct answer even if the weighted graph contains the negative weight values.
- **Rule of this algorithm :-** We will go on relaxing all the edges (n - 1) times where, n = number of vertices.
- **Relaxing means:** If (d(u) + c(u , v) < d(v)) d(v) = d(u) + c(u , v)
- To find the shortest path of above graph, the first step is note down all edges which are given below:

(A, B), (A, C), (A, D), (B, E), (C, E), (D, C), (D, F), (E, F), (C, B)

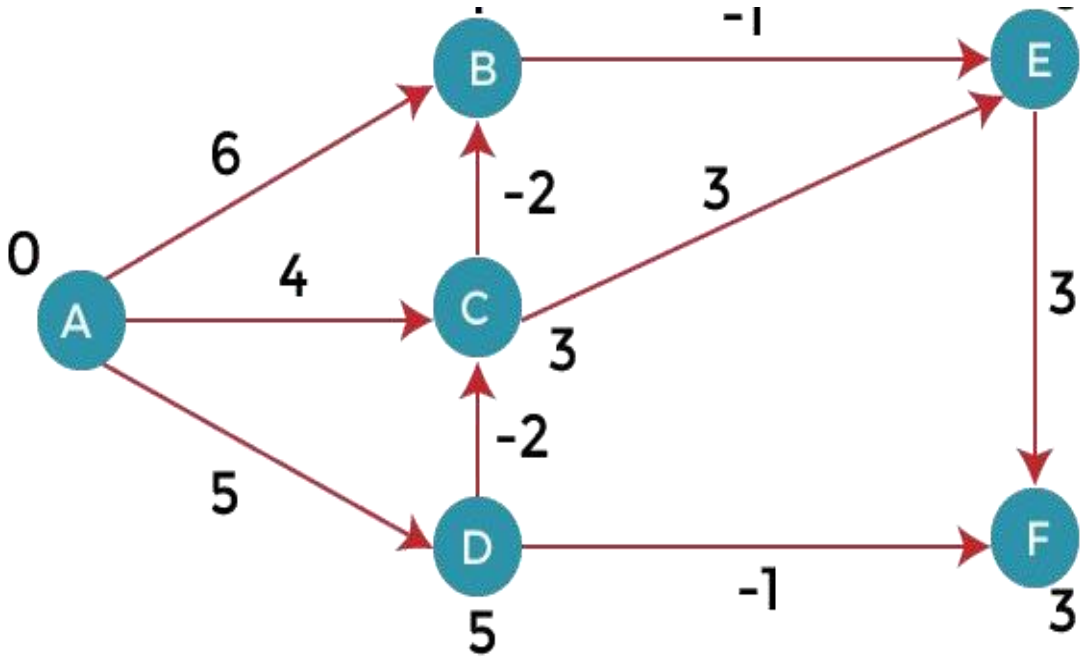


BELLMAN FORD ALGORITHM

- Let's consider the source vertex as 'A'; therefore, the distance value at vertex A is 0 and the distance value at all the other vertices as infinity shown as below:
- Since the graph has six vertices so it will have five iterations.
- **First Iteration:-**



Second iteration:



- We will perform the same steps as we did in the previous iterations. We will observe that there will be no updation in the distance of vertices.

BELLMAN FORD ALGORITHM

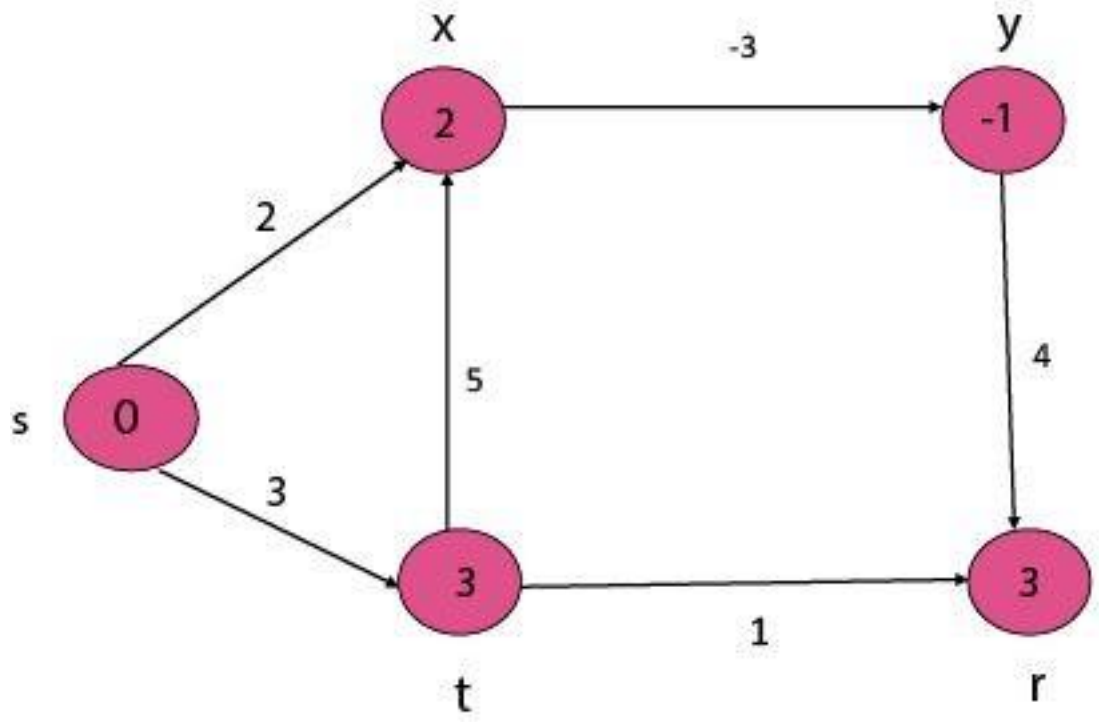
```
BELLMAN-FORD(G, w, s)
INITIALIZE-SINGLE-SOURCE(G, s)
for i = 1 to |G.V| - 1
    for each edge (u, v) ∈ G.E
        RELAX(u, v, w)
for each edge (u, v) ∈ G.E
    if v.d > u.d + w(u, v)
        return FALSE
return TRUE
```

SINGLE SOURCE SHORTEST PATH IN DAG

SINGLE SOURCE SHORTEST PATH IN DIRECTED ACYCLIC PATH

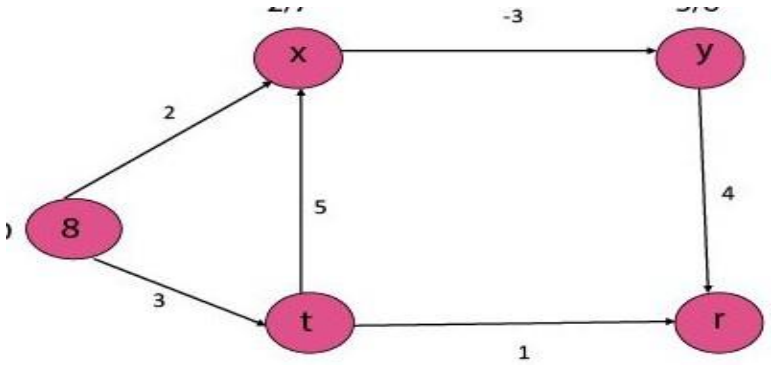
- By relaxing the edges of a weighted DAG (Directed Acyclic Graph) G = (V, E) according to a topological sort of its vertices, we can figure out shortest paths from a single source in Ø(V+E) time. Shortest paths are always well described in a dag, since even if there are negative-weight edges, no negative-weight cycles can exist.

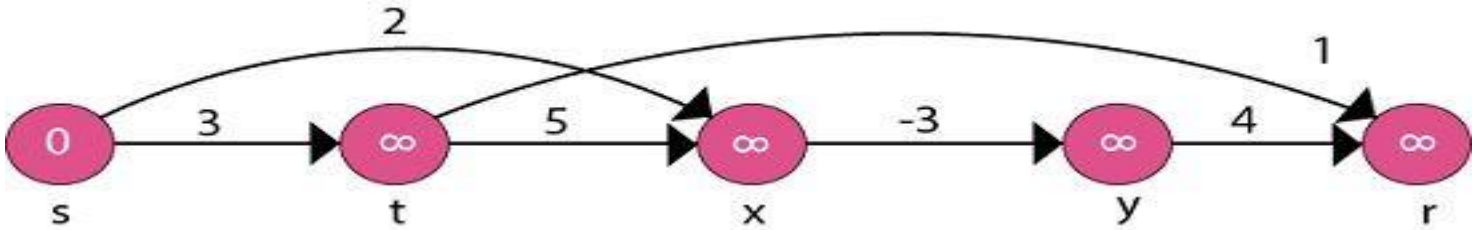
```
DAG-SHORTEST-PATHS(G, w, s)
topologically sort the vertices of G
INITIALIZE-SINGLE-SOURCE(G, s)
for each vertex u, taken in topologically sorted order
    for each vertex v ∈ G.Adj[u]
        RELAX(u, v, w)
```



SINGLE SOURCE SHORTEST PATH IN DIRECTED ACYCLIC PATH

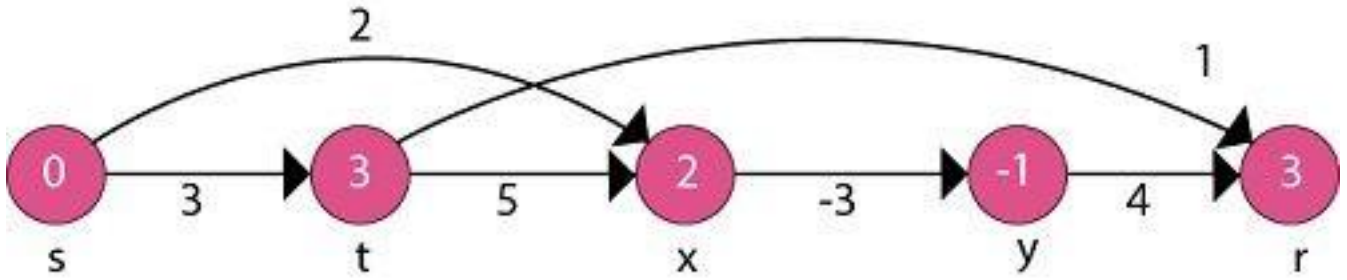
- **Step1:** To topologically sort vertices apply **DFS (Depth First Search)** and then arrange vertices in linear order by decreasing order of finish time.





Initialize Single Source

- Now, take each vertex in topologically sorted order and relax each edge.



Thus the Shortest Path is:

1. s to x is 2
2. s to y is -1
3. s to t is 3
4. s to r is 3

PROOF OF SHORTEST PATH PROPERTIES

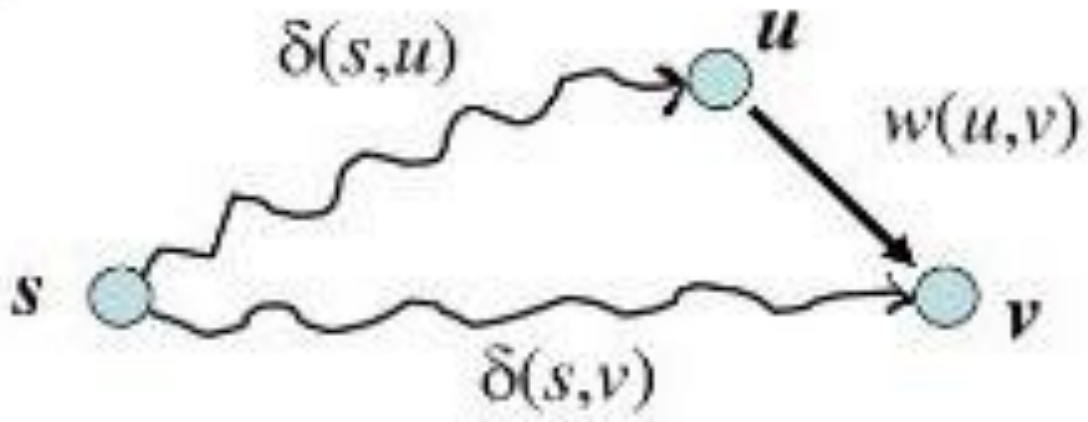
SHORTEST PATH PROPERTIES

- **Triangle inequality**
  - For all $(u, v) \in E$, we have $\delta(s, v) \leq \delta(s, u) + w(u, v)$.

    ***Proof*:**

    Weight of shortest path $s \rightsquigarrow v$ is $\leq$ weight of any path $s \rightsquigarrow v$.

    Path $s \rightsquigarrow u \rightarrow v$ is a path $s \rightsquigarrow v$, and if we use a shortest path $s \rightsquigarrow u$, its weight is $\delta(s, u) + w(u, v)$. ■



SHORTEST PATH PROPERTIES

- **Upper Bound Properties**
-

- Always have $d[v] \geq \delta(s, v)$ for all v.

  Once $d[v] = \delta(s, v)$, it never changes.

  ***Proof*:** Initially true.

  Suppose there exists a vertex such that $d[v] < \delta(s, v)$.

  Without loss of generality, v is first vertex for which this happens.

  Let u be the vertex that causes $d[v]$ to change.

  Then $d[v] = d[u] + w(u, v)$.

  So, $d[v] < \delta(s, v)$

  $\leq \delta(s, u) + w(u, v)$ (triangle inequality)

  $\leq d[u] + w(u, v)$ (v is first violation)

  $\Rightarrow d[v] < d[u] + w(u, v)$. (Contradicts $d[v] = d[u] + w(u, v)$)

  Once $d[v]$ reaches $\delta(s, v)$, it never goes lower. It never goes up, since relaxations only lower shortest-path estimates.

SHORTEST PATH PROPERTIES

- **Upper Bound Property**

► Always have $d[v] \geq \delta(s, v)$ for all v.
Once $d[v] = \delta(s, v)$, it never changes.

***Proof*:** Initially true.
Suppose there exists a vertex such that $d[v] < \delta(s, v)$.
Without loss of generality, v is first vertex for which this happens.
Let u be the vertex that causes $d[v]$ to change.
Then $d[v] = d[u] + w(u, v)$.

$$\begin{aligned} \text{So, } d[v] &< \delta(s, v) \\ &\leq \delta(s, u) + w(u, v) \quad \text{(triangle inequality)} \\ &\leq d[u] + w(u, v) \quad (v \text{ is first violation}) \end{aligned}$$

$\Rightarrow d[v] < d[u] + w(u, v)$. (Contradicts $d[v] = d[u] + w(u, v)$)
Once $d[v]$ reaches $\delta(s, v)$, it never goes lower. It never goes up, since relaxations only lower shortest-path estimates.

- **No-Path Property:-** If $\delta(s, v) = \infty$, then $d[v] = \infty$ always.

***Proof*:**
$d[v] \geq \delta(s, v) = \infty \Rightarrow d[v] = \infty$.

SHORTEST PATH PROPERTIES

- **Convergence Property**

  - If $s \leadsto u \rightarrow v$ is a shortest path, $d[u] = \delta(s, u)$, and we call RELAX(u, v, w), then $d[v] = \delta(s, v)$ afterward.

    ***Proof*:**

    After relaxation:

    $d[v] \leq d[u] + w(u, v)$ (RELAX code)

    $= \delta(s, u) + w(u, v)$

    $= \delta(s, v)$ (lemma ----- optimal substructure)

    Since $d[v] \geq \delta(s, v)$, must have $d[v] = \delta(s, v)$. ■

SHORTEST PATH PROPERTIES

- **Path Relaxation Property**

Let $p = < v_0, v_1, \ldots, v_k >$ be a shortest path from $s = v_0$ to v_k . If we relax, *in order*, $(v_0, v_1), (v_1, v_2), \ldots, (v_{k-1}, v_k)$, even intermixed with other relaxations, then $d[v_k] = \delta(s, v_k)$.

***Proof*:**

Induction to show that $d[v_i] = \delta(s, v_i)$ after (v_{i-1}, v_i) is relaxed.

Basis:

$i = 0$. Initially, $d[v_0] = 0 = \delta(s, v_0) = \delta(s, s)$.

Inductive step:

Assume $d[v_{i-1}] = \delta(s, v_{i-1})$.

Relax (v_{i-1}, v_i).

By Convergence Property, $d[v_i] = \delta(s, v_i)$ afterward and $d[v_i]$ never changes.

THANKS